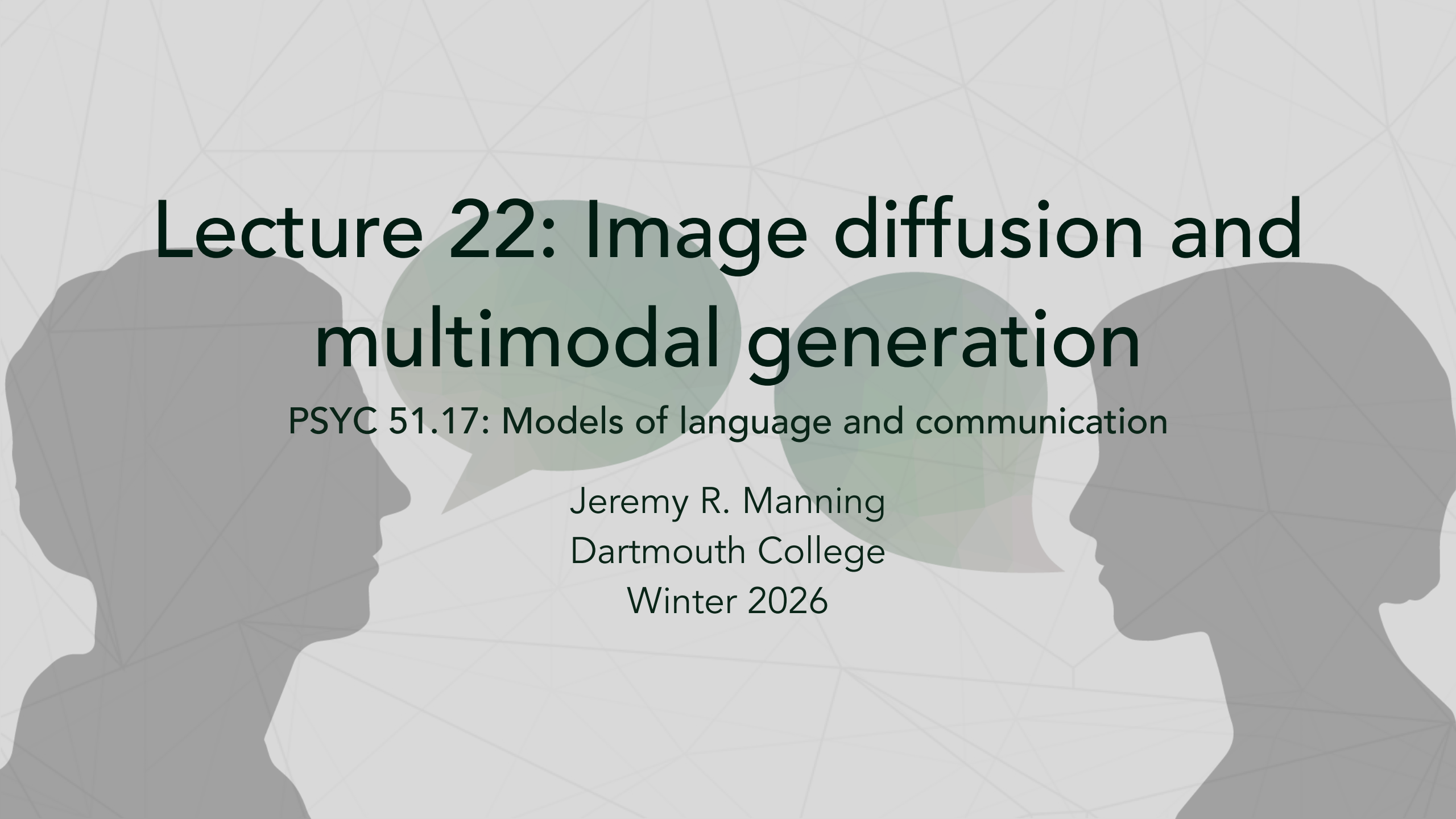




# Lecture 22: Image diffusion and multimodal generation

PSYC 51.17: Models of language and communication

Jeremy R. Manning  
Dartmouth College  
Winter 2026

# Learning objectives

By the end of this lecture, you will be able to...

1. Explain how **latent diffusion** compresses computation via a VAE to enable high-resolution image generation
2. Describe how **CLIP** creates a shared embedding space for text and images
3. Explain how **cross-attention** and **classifier-free guidance** enable text-controlled image generation
4. Compare the **U-Net** and **Transformer (DiT)** backbones for diffusion models
5. Generate images from text prompts using the HuggingFace `diffusers` library

# From text to images

## Bridging Lecture 21 and today

In Lecture 21, we saw how diffusion models generate **text** by iteratively unmasking tokens (discrete diffusion). Today we turn to the domain where diffusion was born — **images** — and ask: how do we generate high-resolution images from text prompts?

## The key challenges

| Challenge                      | Solution (this lecture)                                           |
|--------------------------------|-------------------------------------------------------------------|
| Images are huge (786K+ values) | <b>Latent diffusion</b> — compress first, diffuse in latent space |
| Text must control generation   | <b>CLIP + cross-attention</b> — bind words to spatial regions     |
| Output must match the prompt   | <b>Classifier-free guidance</b> — amplify text alignment          |
| Architecture must scale        | <b>U-Net</b> or <b>DiT</b> — backbones for the denoising network  |

# The pixel problem

## Why pixel-space diffusion is expensive

The DDPM framework (Lecture 21) can be applied to images by adding Gaussian noise to pixels and learning to reverse the process. But for a  $512 \times 512$  RGB image, the denoising network must process  $512 \times 512 \times 3 = 786,432$  values at every step — far too expensive for high-resolution generation.

## U-Net (brief preview)

The **U-Net** ([Ronneberger et al., 2015](#)) is an encoder-decoder neural network with skip connections, originally designed for image segmentation. It became the default backbone for diffusion models — we'll cover it in detail later this lecture.

## The numbers

| Resolution       | Pixels  | Memory | Time per image |
|------------------|---------|--------|----------------|
| $64 \times 64$   | 12,288  | ~2 GB  | ~30 seconds    |
| $256 \times 256$ | 196,608 | ~8 GB  | ~5 minutes     |
| $512 \times 512$ | 786,432 | ~32 GB | ~20 minutes    |

The solution: don't run diffusion in pixel space. Run it in a **compressed latent space**.

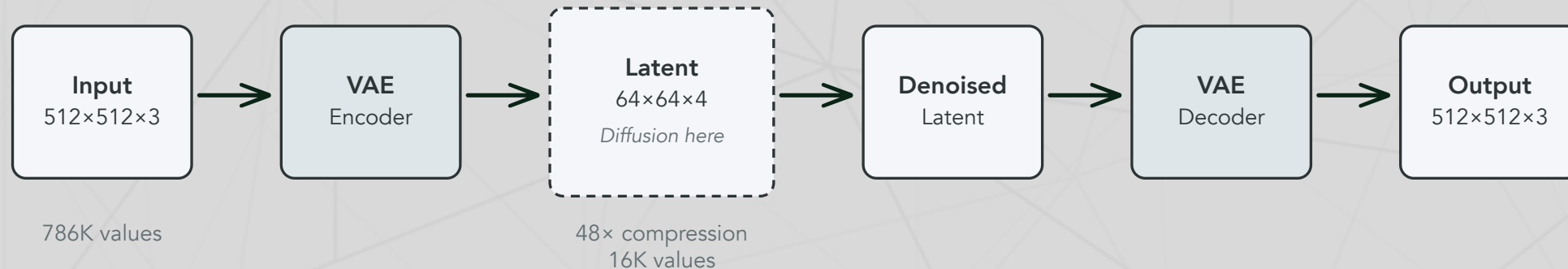
# Latent diffusion

## Compressing before diffusing

Latent diffusion separates image generation into two stages:

1. **Compression:** A pretrained **variational autoencoder** (VAE) encodes images into a compact latent space (typically  $8\times$  spatial compression)
2. **Generation:** The diffusion process operates entirely in this latent space

The VAE is described on the next slide.



## Further reading

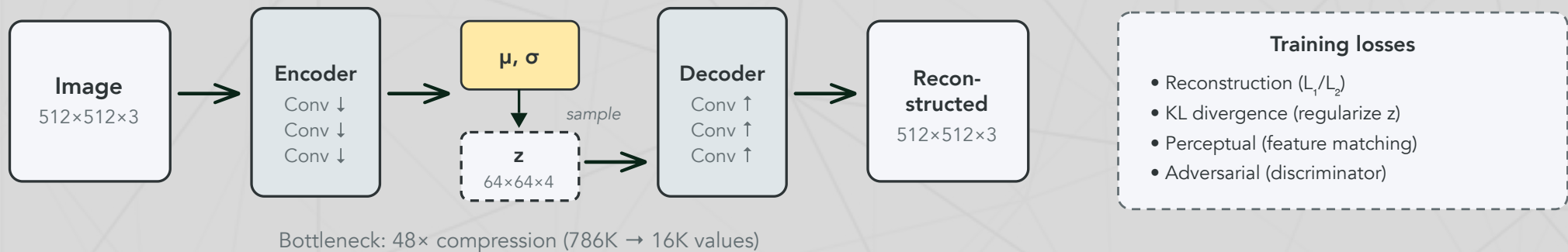
**Rombach et al. (2022, CVPR)**. "High-resolution image synthesis with latent diffusion models" — The paper that introduced latent diffusion and enabled Stable Diffusion.

# Variational autoencoder (VAE)

## Compressing images to a latent space

A **VAE** learns to compress images into a low-dimensional latent representation and reconstruct them:

1. **Encoder:** Maps image  $\mathbf{x} \in \mathbb{R}^{512 \times 512 \times 3}$  to mean  $\boldsymbol{\mu}$  and variance  $\boldsymbol{\sigma}^2$  of a latent distribution
2. **Sampling:** Draw  $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$  — a latent vector in  $\mathbb{R}^{64 \times 64 \times 4}$
3. **Decoder:** Reconstructs the image from  $\mathbf{z}$



## Why this works

The VAE learns to preserve perceptually important information while discarding redundant detail. Diffusion in latent space is **~50× cheaper** than in pixel space, with negligible quality loss. This single insight enabled Stable Diffusion — the first open-source, consumer-GPU image generator.



# The VAE bottleneck

## Stable Diffusion's VAE in numbers

| Property               | Value                                      |
|------------------------|--------------------------------------------|
| Input resolution       | $512 \times 512 \times 3$                  |
| Latent resolution      | $64 \times 64 \times 4$                    |
| Compression ratio      | $48\times$ (786K $\rightarrow$ 16K values) |
| Reconstruction quality | Near-lossless for natural images           |

The VAE is trained once and frozen. The diffusion model only ever sees latents — it never touches pixels during training or sampling.

## How the VAE is trained

- **Reconstruction loss** ( $L_1/L_2$ ): Ensure decoded image matches the original
- **KL divergence**: Regularize the latent space so it's smooth and continuous
- **Perceptual loss**: Compare *features* extracted by a pretrained network (e.g., VGG), not raw pixels — so the model prioritizes visual similarity over pixel-exact matching
- **Adversarial loss**: A discriminator network tries to distinguish real from reconstructed images, pushing the decoder toward photorealistic outputs

# CLIP: connecting text and images

## Contrastive language-image pre-training

CLIP learns a **shared embedding space** for text and images. It was trained on 400 million image-text pairs from the internet:

1. **Image encoder** (Vision Transformer): Maps an image to a vector
2. **Text encoder** (Transformer): Maps a caption to a vector
3. **Contrastive training**: Matching pairs are pulled close; non-matching pairs are pushed apart

## Why CLIP matters for diffusion

CLIP provides the "language understanding" for text-to-image systems. When you type "a cat wearing a hat," CLIP's text encoder converts it into an embedding that captures the **visual meaning** — not just the linguistic meaning. This embedding then guides diffusion via cross-attention.

## Further reading

Radford et al. (2021, *ICML*) "Learning transferable visual models from natural language supervision" — Connection to Lecture 11: CLIP extends the idea of **learned embeddings** (Word2Vec, GloVe) to a *joint* text-image space.

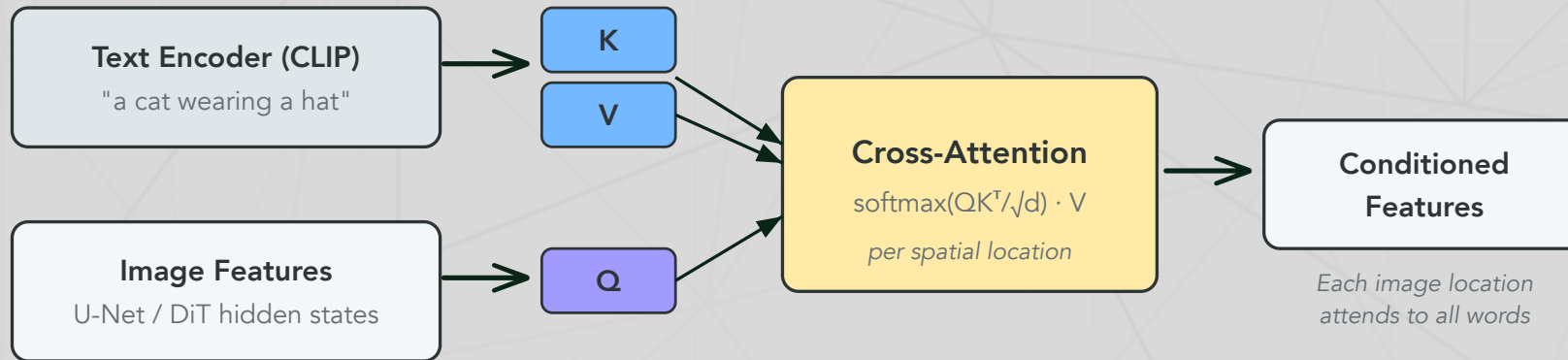


# Text conditioning with cross-attention

## How text controls image generation

To generate images from text, the denoising backbone adds **cross-attention** layers (recall attention from Lecture 15). At each layer:

1. The text prompt is encoded by CLIP into a sequence of token embeddings
2. The image features at that layer serve as **queries** (Q)
3. The text embeddings serve as **keys** (K) and **values** (V)
4. Cross-attention allows each spatial location in the image to attend to relevant words



## How 'a cat wearing a hat' becomes an image

The word "cat" activates high attention weights in the spatial region where the cat is generated. The word "hat" activates weights near the top of the cat region. This spatial-linguistic binding is learned entirely from image-caption pairs.

# Classifier-free guidance

## Controlling generation quality

Classifier-free guidance (CFG) improves alignment between text and generated images. During training, the text condition is randomly dropped some fraction of the time. At inference:

$$\tilde{\epsilon} = \underbrace{\epsilon_{\theta}(\mathbf{x}_t, t, \emptyset)}_{\text{unconditional}} + w \cdot \left( \underbrace{\epsilon_{\theta}(\mathbf{x}_t, t, c)}_{\text{conditional}} - \underbrace{\epsilon_{\theta}(\mathbf{x}_t, t, \emptyset)}_{\text{unconditional}} \right)$$

where  $w$  is the **guidance scale** (typically 7–15).

## The intuition

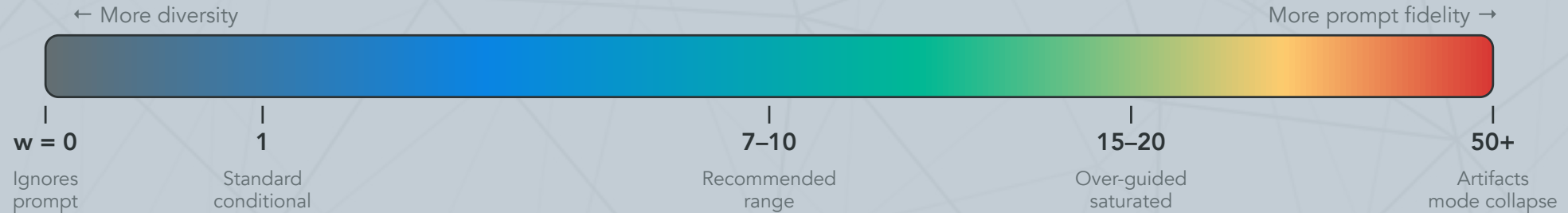
Think of CFG as asking: "What's *different* about images that match this prompt versus random images?" Then amplifying that difference. The model learns both what a "dog on a beach" looks like *and* what a random image looks like — guidance amplifies the gap. See the next slide for examples.

## Further reading

**Ho & Salimans (2022)**. "Classifier-free diffusion guidance" — Used in virtually all modern text-to-image systems.

# The guidance scale tradeoff

## What different guidance scales produce



## In practice

Nearly every text-to-image system uses CFG. Stable Diffusion defaults to  $w = 7.5$ ; DALL-E 2 uses  $w \approx 4$ . The range of 7-10 balances prompt fidelity against visual quality — pushing higher creates oversaturated, artifact-prone images.

## Try it: varying the guidance scale

```
1 from diffusers import StableDiffusionPipeline
2 import torch
3
4 pipe = StableDiffusionPipeline.from_pretrained(
5     "runwayml/stable-diffusion-v1-5", torch_dtype=torch.float16
6 ).to("cuda")
7
8 for scale in [1.0, 7.5, 20.0]:
```

continued...

# The guidance scale tradeoff

Try it: varying the guidance scale

```
9 img = pipe("a cat in a hat", guidance_scale=scale).images[0]  
10 img.save(f"guidance_{scale}.png")
```

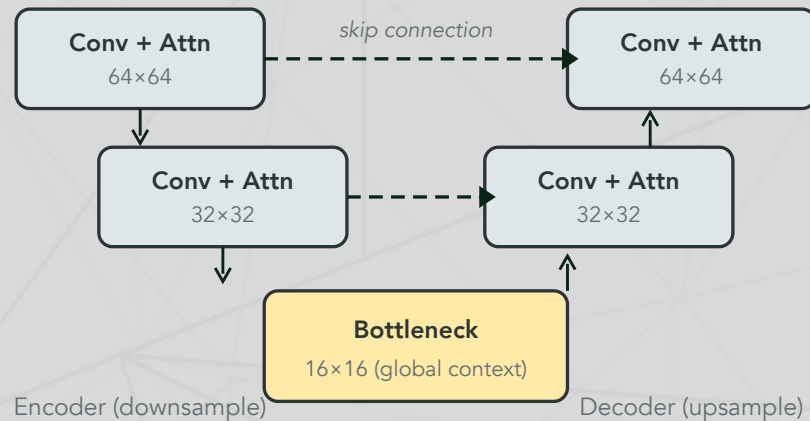
...continued

# U-Net: the original diffusion backbone

## An encoder-decoder with skip connections

The U-Net is the default backbone for diffusion models from DDPM through Stable Diffusion 1–2. Its U-shaped design provides **multi-scale reasoning**:

1. **Encoder** (downsampling): Reduces spatial resolution while increasing channels — captures global context
2. **Bottleneck**: Lowest resolution, largest receptive field
3. **Decoder** (upsampling): Restores spatial resolution — generates fine details
4. **Skip connections**: Connect encoder to decoder at matching resolutions — preserve high-frequency spatial information



### Conditioning inputs

- Timestep  $t$  (sinusoidal embedding)

### Text conditioning

- CLIP embeddings via cross-attention

### Key properties

- Same-size input/output
- Multi-scale reasoning
- Skip connections preserve high-frequency detail
- ~860M params (SD 1.5)

## Further reading

**Ronneberger et al. (2015, *MICCAI*)** "U-Net: convolutional networks for biomedical image segmentation" — Originally for medical imaging, now the workhorse of diffusion.



# U-Net in diffusion models

## How the U-Net is adapted for diffusion

| Property              | Configuration                                       |
|-----------------------|-----------------------------------------------------|
| Input/output          | Same-sized latent (e.g., $64 \times 64 \times 4$ )  |
| Timestep conditioning | Sinusoidal embeddings added to each block           |
| Text conditioning     | CLIP embeddings injected via cross-attention layers |
| Skip connections      | Concatenate encoder features to decoder             |
| Parameters            | ~860M (Stable Diffusion 1.5)                        |

## The intuition: why multi-scale matters

The U-Net's encoder-decoder structure lets it reason at **multiple scales simultaneously**. The bottleneck captures high-level composition ("a dog sits on the left, a tree on the right") while skip connections preserve fine details ("the dog's fur texture, the tree's leaves"). This is why U-Nets produce images with both coherent structure *and* crisp details.

## Connection to Lecture 15

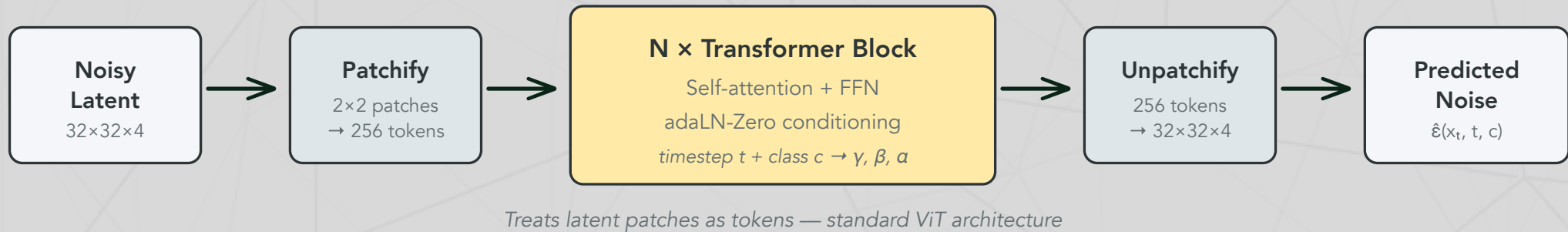
Cross-attention in the U-Net works exactly like encoder-decoder attention in the original Transformer (Lecture 15) — the image features are the "decoder" queries, and the text embeddings are the "encoder" keys and values.

# Diffusion Transformer (DiT)

## Replacing U-Net with a Transformer

The Diffusion Transformer (DiT) replaces the U-Net with a standard Vision Transformer:

1. **Patchify**: Divide the noisy latent into non-overlapping patches (e.g.,  $2 \times 2$ )
2. **Flatten**: Treat patches as a sequence of tokens (just like ViT treats image patches — Lecture 15)
3. **Process**: Apply standard Transformer blocks with self-attention
4. **Unpatchify**: Reshape back to spatial dimensions to get the predicted noise



## Key terms

- **Noisy latent**: The VAE-compressed image with Gaussian noise added at timestep  $t$  — this is the input the denoising network must "clean up"
- **Predicted noise**: The network's estimate of what noise was added — subtract it from the noisy latent to get a cleaner image

# DiT: why replace U-Net?

## Transformers scale better

DiT-XL/2 (675M parameters) achieves state-of-the-art FID of 2.27 on ImageNet, beating all previous diffusion models. More importantly, DiT shows **clean scaling behavior** — larger models consistently produce better results, with no architectural bottlenecks.

## The intuition

U-Nets have strong *inductive biases* for spatial data (locality, hierarchy). These help with small models but become constraints at scale. Transformers make fewer assumptions and let the model learn the right structure from data — the same lesson we saw with language models (Lectures 15–16).

## Further reading

**Peebles & Xie (2023, ICCV)**, "Scalable diffusion models with Transformers" — DiT showed that Transformers can replace U-Nets in diffusion, and the result scales better.

# Try it: generate an image

Text-to-image with Stable Diffusion (Google Colab)

```
1  !pip install -q diffusers transformers accelerate
2
3  import torch
4  from diffusers import StableDiffusionPipeline
5
6  pipe = StableDiffusionPipeline.from_pretrained(
7      "runwayml/stable-diffusion-v1-5", torch_dtype=torch.float16
8  ).to("cuda")
9
10 image = pipe("a photo of an astronaut riding a horse on mars").images[0]
11 image.save("astronaut.png")
```

What's happening under the hood

1. CLIP encodes your text prompt into embeddings
2. The U-Net iteratively denoises a random latent, guided by those embeddings via cross-attention
3. CFG amplifies the text signal at each step (default `guidance_scale=7.5` )
4. The VAE decoder converts the final latent back to a 512×512 pixel image

Every concept from this lecture is working together in those 5 lines of code.

# Take-home messages

## Key ideas from today

- The key bottleneck in high-resolution generation wasn't the diffusion process itself — it was **where** you run it. Compressing to latent space via a VAE made consumer-GPU generation possible.
- Text-to-image generation requires a **bridge between modalities**: CLIP creates a shared embedding space (extending the idea of word embeddings from Lecture 11), and cross-attention lets the diffusion model "listen" to text at every spatial location (using the same mechanism as encoder-decoder attention from Lecture 15).
- Classifier-free guidance shows that **controlling** generation is as important as generation itself — the trick is surprisingly simple: learn what conditional and unconditional outputs look like, then amplify the difference.
- The field evolves by **composing** innovations (VAE + CLIP + CFG + U-Net/DiT), not replacing them. Each addresses one specific limitation.



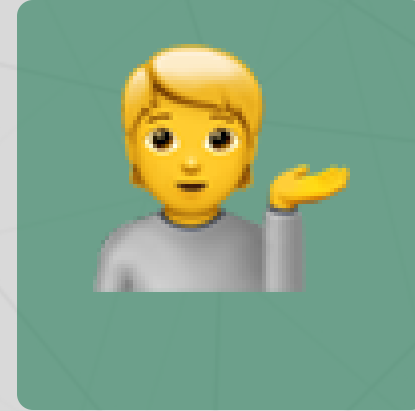
# Questions?



Email



Discord



Office hours

## Up next...

Diffusion applications: text-to-image (DALL-E 2, Imagen), text-to-video (Sora), and the ethics of generative AI